CHAPTER 2

Data, Types, and Variables

When we write software, it is typically used for the transformation and transmission of data so that it can become information. The device that runs our programs (the hardware upon which our systems run) isn't aware of concepts such as human language or the protocols that drive the internet. Data is simply a sequence of bytes: a stream of ones and zeroes. Those byte values only become information when they are evaluated in their proper context. For example, consider the following sequence of bytes.

```
Binary:

01001000 01000101 01001100 01001100 01001111 00100000 01010111 01001111
01010010 01001100 01000100

Hex:

0x48 0x45 0x4C 0x4C 0x4F 0x20 0x57 0x4F 0x52 0x4C 0x44
```

This is raw data, and I bet you're having a hard time extracting information from it, right? Those 1's and 0's are relatively meaningless to humans, as are the corresponding hexadecimal values unless we know what they're for. (If you're unsure what a hexadecimal number is, Chapter 7, "Numbers," explains about different number bases.) If we decide to convert them to text using an encoding system called ASCII (American Standard Code for Information Interchange – which has been around even longer than I have) – then information emerges.

ASCII is a system for encoding characters into numbers; each numerical byte value corresponds to a character, and the table of byte value to character mappings is called a *codepage*. More modern codepages (such as Unicode) use longer byte sequences, but the most widely used (UTF-8) is a superset of ASCII (in order to preserve compatibility with older systems). We will examine the handling of text data in greater detail in a later chapter.

© Paul Tregoing 2025
P. Tregoing, *Exploring PHP 8*, https://doi.org/10.1007/979-8-8688-2136-3_2